

Golang 泛型

主讲教师：（大地）

合作网站：www.itying.com （IT 营）

我的专栏：<https://www.itying.com/category-79-b0.html>

一、泛型介绍	1
二、泛型的使用	3

一、泛型介绍

Golang 在 1.18 版本（2022 年 3 月发布）引入泛型特性。Go 泛型解决了函数、方法、结构体和接口的复用性问题，并在编译时提供对不特定数据类型的类型安全支持。

我想调用方法可以传入不同的数据类型，下面是实现代码

```
package main

import "fmt"

func getStringData(value string) string {
    return value
}

func getIntData(value int) int {
    return value
}

func getBoolData(value bool) bool {
    return value
}

func main() {
    // 使用特定类型的函数
    str := getStringData("this is str")
}
```

```
fmt.Println(len(str)) // 输出: 11

num := getIntData(123)

fmt.Println(num)

b := getBoolData(true)

fmt.Println(b)

}
```

问题:

- 1、函数重复: 三个函数 `getStringData`、`getIntData`、`getBoolData` 功能完全相同, 只是参数和返回类型不同
- 2、重复的代码增加了维护成本
- 3、虽然类型安全, 但代码冗余

Interface 接口改进方案

```
package main

import "fmt"

func getDataInterface(value interface{}) interface{} {

    return value

}

func main() {

    str1 := getDataInterface("hello")

    fmt.Println(str1)

    num2 := getDataInterface(123)

    fmt.Println(num2)

}
```

问题:

- 1、解决了函数重复问题, 但是带来了新的问题就是**类型安全问题**
- 2、`getDataInterface` 函数返回 `interface{}` 类型, 调用者需要自己进行类型断言才能安全使用

二、泛型的使用

1、类型参数

使用方括号 [] 声明类型参数: [T any] 声明了一个类型参数 T, any 表示 T 可以是任何类型。函数可以接受和返回任意类型的值

```
package main

import "fmt"

func getData[T any](value T) T {
    return value
}

func main() {
    str1 := getData("hello")
    fmt.Println(str1) // str1 是 string 类型

    num2 := getData(123)
    fmt.Println(num2) // num2 是 int 类型
}
```

2、使用接口定义类型约束

```
package main

import "fmt"

type Number interface {
    int | int64 | float64
}

func Add[T Number](a, b T) T {
```

```
    return a + b
}

func main() {

    fmt.Println(Add(1, 2))

    fmt.Println(Add(1.1, 2.2))

    // fmt.Println(Add("11", "32432")) //错误
}
```

3、泛型结构体（一）

```
package main

import "fmt"

// 泛型结构体
type Container[T any] struct {

    value T
}

// 泛型方法
func (c *Container[T]) Set(value T) {

    c.value = value
}

func (c *Container[T]) Get() T {

    return c.value
}

func main() {

    // 使用 int 类型

    intContainer := Container[int]{}

    intContainer.Set(42)
```

```
fmt.Println("Int value:", intContainer.Get()) // 输出: 42

// 使用 string 类型

strContainer := Container[string]{}

strContainer.Set("Hello, Generics!")

fmt.Println("String value:", strContainer.Get()) // 输出: Hello, Generics!

}
```

4、泛型结构体 （二）

~int 匹配底层类型为 int 的所有类型，包括 int 本身、类型别名和自定义类型

```
package main

import (
    "fmt"
)

// 自定义约束
type Ordered interface {
    ~int | ~int8 | ~int16 | ~int32 | ~int64 |
        ~uint | ~uint8 | ~uint16 | ~uint32 | ~uint64 | ~uintptr |
        ~float32 | ~float64 |
        ~string
}

// 泛型类
type MinClass[T Ordered] struct {
    list []T
}
```



```
// 添加方法

func (m *MinClass[T]) Add(list []T) {

    m.list = list

}

// 求最小值方法

func (m *MinClass[T]) Min() T {

    if len(m.list) == 0 {

        var zero T

        return zero

    }

    tempMin := m.list[0]

    for i := 1; i < len(m.list); i++ {

        if m.list[i] < tempMin {

            tempMin = m.list[i]

        }

    }

    return tempMin

}

func main() {

    // 字符串类型

    obj := &MinClass[string]{}

    obj.Add([]string{"1", "2", "3"})

    fmt.Println(obj.Min())

}
```

```
// 整数类型示例

objInt := &MinClass[int]{}

objInt.Add([]int{8, 2, 3, 5, 1})

fmt.Println(objInt.Min())

}
```

5、泛型接口

```
package main

import "fmt"

// 泛型接口
type Usber[T any] interface {

    Start()

    Stop()

    GetDevice() T // 添加一个获取具体设备的方法
}

type Phone struct {

    Name string
}

func (p Phone) Start() {

    fmt.Println(p.Name, "开始工作")
}
```



```
func (p Phone) Stop() {  
    fmt.Println("phone 停止")  
}  
  
func (p Phone) GetDevice() Phone {  
    return p  
}  
  
type Camera struct {  
    Brand string  
}  
  
func (c Camera) Start() {  
    fmt.Println(c.Brand, "相机 开始工作")  
}  
  
func (c Camera) Stop() {  
    fmt.Println(c.Brand, "相机 停止工作")  
}  
  
func (c Camera) GetDevice() Camera {  
    return c  
}  
  
// 电脑结构体 - 使用泛型  
type Computer[T any] struct {
```



```
Name string
}

// 电脑的 Work 方法使用泛型接口
func (c Computer[T]) Work(usb Usber[T]) {

    usb.Start()

    usb.Stop()


    // 可以通过 GetDevice 获取具体的设备信息

    device := usb.GetDevice()

    fmt.Printf("设备信息: %+v\n", device)
}

func main() {

    phone := Phone{

        Name: "小米手机",

    }


    camera := Camera{

        Brand: "佳能",

    }


    // 使用泛型电脑

    computer := Computer[Phone]{Name: "我的电脑"}

    computer.Work(phone)


    // 对于不同类型的设备，需要创建对应的电脑实例
```



```
computer2 := Computer[Camera]{Name: "我的电脑"}  
  
computer2.Work(camera)  
  
fmt.Println("\n--- 使用通用函数 ---")  
  
}
```